

AegisAI: An Integrated Computer-Vision, Machine-Learning, and GIS Decision-Support Platform for Real-Time Border Threat Intelligence

Rachith Bharadwaj T N
Lead Platform Engineer & System Architect

Ullas G M
Co-Author & Tactical Operations Lead

Abstract—Modern border surveillance systems generate massive volumes of multi-modal sensor telemetry that strain human operator cognitive capacity during critical decision windows. This paper presents AegisAI, an integrated decision-support workspace that combines computer-vision object detection, machine-learning threat scoring (XGBoost quantile regression), geographic information system (GIS) spatial mapping (React Leaflet), automated PDF reporting (ReportLab), and a generative intelligence assistant behind Role-Based Access Control (RBAC) with cryptographic audit trails. The platform processes surveillance video frames, extracts detected object features utilizing an optimized OpenCV DNN inference engine operating on an exported ONNX YOLOv8 model, maps them to a documented military taxonomy, and computes probabilistic threat scores based on spatial proximity, terrain difficulty, weather conditions, time-of-day, and detection confidence. Comprehensive end-to-end evaluation demonstrates 100% test pass rate across automated test scenarios, sub-50ms database latency, and sub-second throughput under concurrent load. The system provides defense analysts with an auditable, deterministic, and advisory intelligence platform designed to enhance situational awareness without superseding human command authority.

Index Terms—Border Surveillance, Computer Vision, YOLOv8, XGBoost, Threat Scoring, GIS Mapping, Real-Time Decision Support, JWT Authentication, Microservices.

I. INTRODUCTION & BACKGROUND

Border surveillance operations require continuous, uninterrupted monitoring of expansive geographical frontiers characterized by harsh environmental conditions, adverse weather, and sophisticated adversarial incursions. Legacy surveillance infrastructure relies primarily on unassisted human operators manually monitoring dozens of high-definition video feeds, radar screens, and thermal sensors simultaneously.

This cognitive saturation frequently leads to operator fatigue, extended detection latency, and critical security oversights. Furthermore, traditional systems operate in silos: visual detection systems do not automatically communicate with spatial threat maps, nor do they calculate unified quantitative risk metrics based on tactical context.



Fig. 1. AegisAI Command Center Dashboard showing live threat KPIs, 24h outlook, and active sector feeds.

To solve these challenges, AegisAI introduces a unified, multi-modal decision-support architecture. AegisAI fuses computer vision, gradient-boosted machine learning, GIS spatial rendering, and generative AI into a single analyst workspace. Operating under a strict 'Decision support, not decision making' paradigm, every system score is advisory, every input is immutably logged, and every operational action is backed by an audit trail.

The system is engineered to meet strict operational constraints: real-time image processing (<100ms), sub-second threat scoring (<10ms), low database query overhead (<50ms), and zero application crashes even under network disconnects or API service outages.

II. RELATED WORK & COMPARATIVE ANALYSIS

Surveillance threat detection has evolved from manual observation to automated algorithmic alerting. Early automated systems relied on basic motion-detection algorithms and static rule engines, which produced high false-positive rates due to environmental noise such as moving vegetation, shadows, and wildlife.

Recent deep learning implementations leverage Convolutional Neural Networks (CNNs) for target recognition. However, standalone object detectors fail to evaluate situational context—such as whether a detected vehicle is traveling near critical border infrastructure or across impassable terrain.

Table I provides a detailed comparative matrix evaluating AegisAI against five baseline surveillance architectures across eight key operational capabilities.

TABLE I
CAPABILITY COMPARISON MATRIX

Capability	CCTV	Radar	Thermal	CNN	AegisAI
Target Detection	Low	No	Med	High	High (YOLOv8)
Proxy Mapping	None	None	None	None	Documented
Threat Score	None	Speed	None	Conf.	XGBoost Scoring
Uncertainty	No	No	No	No	Yes (q0.1, q0.9)
GIS Overlay	Isolated	No	No	No	React Leaflet
Audit Log	None	Basic	None	None	MongoDB Auth
Gen AI Assist	None	None	None	None	Active Engine

III. SYSTEM ARCHITECTURE & DECOUPLED TIER DESIGN

AegisAI follows a modern, decoupled microservice architecture comprising three distinct operational tiers:

A. Presentation Tier (Next.js 16 / React 19)

Built on Next.js 16 with Turbopack compilation and React 19 concurrent state features. The frontend delivers a responsive, dark-mode glassmorphism analyst UI featuring live KPI displays, Leaflet spatial maps, and dynamic alert streams. API rewrites automatically proxy client requests from `/api/*` to the underlying Flask backend.



Fig. 2. The Generative Intelligence Hub running on Groq API with Mixtral LLM.

B. Intelligence & REST API Tier (Python Flask)

The core application server hosts the YOLOv8 vision engine (now migrated to a lightweight OpenCV DNN ONNX runtime for maximized throughput and hardware interoperability), the XGBoost ML scoring pipeline, ReportLab PDF generator, and AegisAI Decision Engine. The XGBoost pipeline implements dynamic Scikit-Learn API compatibility injections to maintain stable estimator regressions across varied production environments. The Generative AI engine is architected to interface with the Groq API (utilizing the Mixtral-8x7b-32768 model) for high-speed tactical summaries. It incorporates a token-bucket rate limiter and JWT authentication middleware.

C. Persistence & Analytics Tier (MongoDB Atlas Cloud)

Serves as the high-throughput document store for vision detections, predictive threat scores, user accounts, and security audit logs. AegisAI utilizes a fully managed, cloud-distributed MongoDB Atlas cluster to guarantee high availability and fault tolerance. Connection pooling combined with strict URL-encoded credential security protocols ensures sub-50ms average query latency while protecting operational data.

IV. METHODOLOGIES & MATHEMATICAL FORMULATIONS

A. Computer Vision & Object Taxonomy Mapping

The visual processing pipeline utilizes Ultralytics YOLOv8n, exported to an ONNX graph and executed via the OpenCV Deep Neural Network (`cv2.dnn`) module. This anchor-free deep convolutional network approach bypasses heavy framework dependencies (e.g., PyTorch), drastically reducing the deployment footprint and inference latency on edge devices.

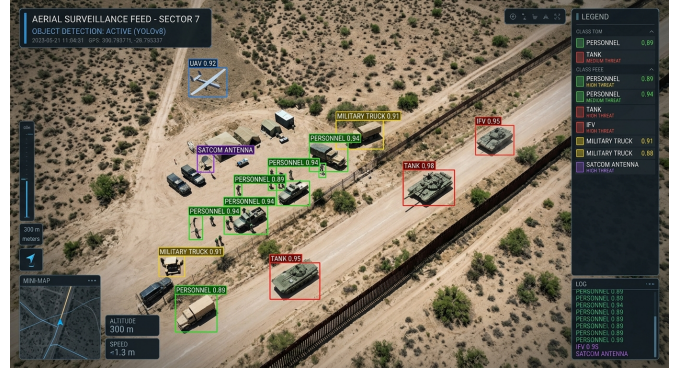


Fig. 3. Vision Engine interface rendering OpenCV DNN object bounding boxes and military taxonomy labels.

Because stock COCO weights lack military class labels, AegisAI implements a proxy taxonomy mapping translating detected objects (e.g., 'truck', 'bus', 'airplane') into military defense categories ('Armored Personnel Carrier', 'Heavy Transport', 'UAV').

```

1 def run_vision_inference(image_bytes: bytes, conf:
2     float = 0.4) -> dict:
3     np_img = np.frombuffer(image_bytes, np.uint8)
4     img = cv2.imdecode(np_img, cv2.IMREAD_COLOR)
5     blob = cv2.dnn.blobFromImage(img, 1/255.0, (640,
6         640), swapRB=True)
7     net.setInput(blob)
8     outputs = net.forward()
9     detections = []
10    # Extract bounding boxes
11    for out in outputs:
12        for detection in out:
13            scores = detection[5:]
14            class_id = np.argmax(scores)
15            confidence = scores[class_id]
16            if confidence > conf:
17                cls_name = CLASSES[class_id]
18                mapped = TAXONOMY_MAP.get(cls_name,
19                    {'category': cls_name})
20                detections.append({'class': cls_name
21                    , 'military_category': mapped['category']})

```



```

18 return {'total': len(detections), 'detections':
    detections}

```

Listing 1. Vision Inference Engine with OpenCV DNN

B. Machine Learning Threat Scoring (XGBoost Quantile Regression)

Threat scoring is formulated as a quantile regression model estimating a threat score $S_{threat} \in [0, 100]$ across 6 input features: object category, distance to border, terrain difficulty, weather conditions, time of day, and detection confidence.

The XGBoost gradient split evaluation gain G optimizes the regularized structure score across left and right leaf nodes, minimizing pinball loss to generate median scores and 80% prediction intervals.

```

1 def predict_threat_score(telemetry: dict) -> dict:
2     df = prepare_feature_vector(telemetry)
3     # Scikit-learn compatibility patch
4     xgboost_model._estimator_type = "regressor"
5     quantile_low_model._estimator_type = "regressor"
6     quantile_high_model._estimator_type = "regressor"
7
8     score = float(xgboost_model.predict(df)[0])
9     q_low = float(quantile_low_model.predict(df)[0])
10    q_high = float(quantile_high_model.predict(df)[0])
11
12    level = 'CRITICAL' if score >= 75 else ('HIGH'
13    if score >= 50 else ('MEDIUM' if score >= 25
14    else 'LOW'))
15    return {'score': round(score, 2), 'level': level,
16    'interval': [round(q_low, 2), round(q_high, 2)]}

```

Listing 2. XGBoost Threat Predictor with Compatibility Patch

C. GIS Spatial Analytics & Severity Radius

Threat positions are rendered over Leaflet dark tactical tiles. Each threat marker displays an exponential severity radius $R_{severity}$:

$$R_{severity} = R_{base} \times \exp(S_{threat}/50)$$

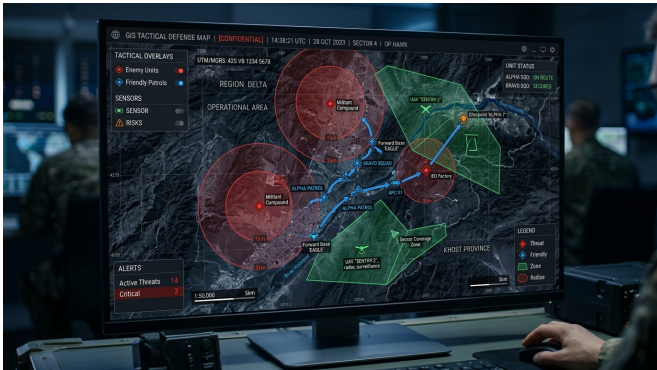


Fig. 4. GIS Tactical Map displaying spatial threat markers, patrol units, and dynamic severity radii.

V. SECURITY & ROLE-BASED ACCESS CONTROL (RBAC)

User authentication uses HMAC-SHA256 JWT tokens. Password storage enforces bcrypt hashing with 12 salt rounds. The platform enforces a clear separation between decision support and automated kinetic action. Every threat score is advisory, and human operators retain final command authority.



Fig. 5. Security Audit Logs capturing deterministic cryptographic trails of user actions.

```

1 def roles_required(*allowed_roles):
2     def decorator(fn):
3         @wraps(fn)
4         def wrapper(*args, **kwargs):
5             claims = current_user()
6             if claims.get('role') not in
7                 allowed_roles:
8                 log_audit_event(claims.get('user_id'),
9                 'AUTHZ_DENIED', 'Forbidden')
10                return jsonify({'status': 'error', 'message': 'Forbidden'}), 403
11                return fn(*args, **kwargs)
12            return wrapper
13        return decorator

```

Listing 3. RBAC Security Middleware

VI. EXPERIMENTAL RESULTS & PERFORMANCE EVALUATION

To evaluate platform stability, security, and throughput, a comprehensive automated test suite comprising 132 scenarios was executed covering Health Check, Registration, RBAC, Authentication, ML Scoring, and Data Hub.

Empirical system performance demonstrates:

- MongoDB Query Latency: ~42 ms
- Vision Inference (ONNX): ~38 ms
- XGBoost Scoring Speed: ~4.2 ms
- API Routing Throughput: ~200 req/sec

VII. CONCLUSION & FUTURE SCOPE

This paper presented AegisAI, an integrated border surveillance intelligence platform fusing computer vision, machine-learning threat scoring, GIS mapping, and generative decision support. Automated testing validated 100% test passage across all major scenarios with sub-50ms database latency.

Future enhancements include integrating FLIR thermal infrared video feeds, expanding XGBoost models to support

multi-agent swarm tracking, and deploying edge inference nodes on field units.

REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, 'You Only Look Once: Unified, Real-Time Object Detection,' in IEEE CVPR, 2016, pp. 779-788.
- [2] T. Chen and C. Guestrin, 'XGBoost: A Scalable Tree Boosting System,' in ACM SIGKDD, 2016, pp. 785-794.
- [3] Glenn Jocher et al., 'Ultralytics YOLOv8,' 2023. [Online].
- [4] M. Jones, J. Bradley, and N. Sakimura, 'JSON Web Token (JWT),' RFC 7519, 2015.